INITIATION PYTHON · JOUR 5 SUR 5 · VENDREDI 6 NOVEMBRE

Vendredi · Algorithmes classiques et code propre

Il y a deux semaines, vous avez écrit en bash la recherche dichotomique, des tris et des fonctions récursives. Aujourd'hui, on les réécrit en Python : le code est deux fois plus court, et on voit enfin l'algorithme au lieu de la syntaxe. L'après-midi, on apprend à ranger son code en plusieurs fichiers, et à chercher un bug avec le débogueur de VS Code plutôt qu'à coups de `print`.

1. La recherche dichotomique

Hier, au juste prix, la meilleure stratégie consistait à proposer le milieu, puis le milieu de la moitié qui reste, et ainsi de suite. C'est la **recherche dichotomique**. Elle ne marche que sur une liste **triée**.

dichotomie.py

```python
def recherche_lineaire(liste, valeur):
    """Renvoie l'indice de valeur et le nombre de comparaisons."""
    for i in range(len(liste)):
        if liste[i] == valeur:
            return i, i + 1
    return -1, len(liste)


def recherche_dichotomique(liste, valeur):
    """Même chose, sur une liste triée, en coupant en deux à chaque tour."""
    debut = 0
    fin = len(liste) - 1
    etapes = 0
    while debut <= fin:
        etapes += 1
        milieu = (debut + fin) // 2
        if liste[milieu] == valeur:
            return milieu, etapes
        elif liste[milieu] < valeur:
            debut = milieu + 1      # la valeur est dans la moitié droite
        else:
            fin = milieu - 1        # la valeur est dans la moitié gauche
    return -1, etapes


nombres = list(range(3, 3001, 3))   # 3, 6, 9, ..., 3000 : 1 000 nombres triés
for cherche in [3, 1500, 2997, 1000]:
    print(f"{cherche} : linéaire {recherche_lineaire(nombres, cherche)}, "
          f"dichotomique {recherche_dichotomique(nombres, cherche)}")
```

Dans le terminal

```
$ python3 dichotomie.py
3 : linéaire (0, 1), dichotomique (0, 9)
1500 : linéaire (499, 500), dichotomique (499, 1)
2997 : linéaire (998, 999), dichotomique (998, 9)
1000 : linéaire (-1, 1000), dichotomique (-1, 10)
```

Sur 1 000 nombres, la recherche linéaire fait jusqu'à 1 000 comparaisons, la dichotomique **jamais plus de 10**. Sur un million, ce serait un million contre 20. À chaque tour, la zone de recherche est divisée par deux : on dit que la complexité est logarithmique.

2. Trier

Le tri par sélection

L'idée : on cherche le plus petit élément et on le met en première place, puis le plus petit du reste en deuxième place, et ainsi de suite.

tri_selection.py

```python
def tri_selection(liste):
    """Trie la liste sur place, du plus petit au plus grand."""
    n = len(liste)
    for i in range(n - 1):
        # on cherche le plus petit entre i et la fin
        i_min = i
        for j in range(i + 1, n):
            if liste[j] < liste[i_min]:
                i_min = j
        # on l'échange avec la case i
        liste[i], liste[i_min] = liste[i_min], liste[i]


notes = [12, 8, 15, 10, 17, 6]
tri_selection(notes)
print(notes)
```

Dans le terminal

```
$ python3 tri_selection.py
[6, 8, 10, 12, 15, 17]
```

La ligne `liste[i], liste[i_min] = liste[i_min], liste[i]` échange deux valeurs en une seule instruction. En bash, il fallait une variable temporaire.

Ce que Python fait déjà

Dans l'interpréteur Python

```
>>> notes = [12, 8, 15, 10, 17, 6]
>>> sorted(notes)
[6, 8, 10, 12, 15, 17]
>>> notes
[12, 8, 15, 10, 17, 6]
>>> notes.sort()
>>> notes
[6, 8, 10, 12, 15, 17]
>>> sorted(["poire", "abricot", "kiwi"])
['abricot', 'kiwi', 'poire']
>>> sorted(["poire", "abricot", "kiwi"], key=len)
['kiwi', 'poire', 'abricot']
>>> sorted(notes, reverse=True)
[17, 15, 12, 10, 8, 6]
```

`sorted(liste)` renvoie une **nouvelle** liste triée et laisse l'ancienne intacte. `liste.sort()` trie la liste **sur place** et ne renvoie rien. `key=len` trie selon la longueur, `reverse=True` du plus grand au plus petit.

Pourquoi écrire son propre tri, alors ? Pour comprendre ce que coûte un algorithme. Mesurons :

chrono.py

```python
import random
import time


def tri_selection(liste):
    n = len(liste)
    for i in range(n - 1):
        i_min = i
        for j in range(i + 1, n):
            if liste[j] < liste[i_min]:
                i_min = j
        liste[i], liste[i_min] = liste[i_min], liste[i]


for taille in [1000, 2000, 4000]:
    nombres = [random.randint(1, 100000) for _ in range(taille)]

    copie = nombres[:]
    debut = time.perf_counter()
    tri_selection(copie)
    t_selection = time.perf_counter() - debut

    debut = time.perf_counter()
    sorted(nombres)
    t_sorted = time.perf_counter() - debut

    print(f"{taille} nombres : sélection {t_selection:.3f} s, sorted {t_sorted:.5f} s")
```

Un exemple de mesure (les temps varient d'une machine à l'autre)

```
$ python3 chrono.py
1000 nombres : sélection 0.013 s, sorted 0.00011 s
2000 nombres : sélection 0.063 s, sorted 0.00024 s
4000 nombres : sélection 0.207 s, sorted 0.00050 s
```

Quand la taille double, le tri par sélection prend **quatre fois** plus de temps : deux boucles imbriquées, la complexité est quadratique. `sorted` utilise un algorithme bien plus malin, et reste presque instantané.

(La ligne `[random.randint(1, 100000) for _ in range(taille)]` fabrique une liste de nombres au hasard en une ligne. C'est une compréhension de liste, on y revient à la section 5.)

3. La récursivité

Une fonction **récursive** s'appelle elle-même, sur un problème plus petit, jusqu'à un cas si simple qu'on connaît la réponse directement : le **cas de base**.

recursif.py

```python
def factorielle(n):
    if n <= 1:                          # cas de base
        return 1
    return n * factorielle(n - 1)       # appel sur un problème plus petit


def rebours(n):
    if n == 0:
        print("Décollage !")
        return
    print(n)
    rebours(n - 1)


print(factorielle(5))
print(factorielle(25))
rebours(3)
```

Dans le terminal

```
$ python3 recursif.py
120
15511210043330985984000000
3
2
1
Décollage !
```

`factorielle(5)` vaut `5 × factorielle(4)`, qui vaut `4 × factorielle(3)`… jusqu'à `factorielle(1)`, qui vaut 1. Puis tout remonte : 1, 2, 6, 24, 120. Et Python calcule sans peine `factorielle(25)`, là où bash débordait dès 21 : ses nombres entiers s'arrêtent vers 9 milliards de milliards, ceux de Python n'ont pas de limite.

Sans cas de base, la fonction s'appellerait à l'infini. Python l'arrête au bout de 1 000 appels environ, avec une `RecursionError`.

Fibonacci, ou l'art de refaire mille fois le même calcul

fibonacci.py

```
appels = 0


def fib(n):
    global appels
    appels += 1
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)


memo = {}


def fib_memo(n):
    global appels
    appels += 1
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memo(n - 1) + fib_memo(n - 2)
    return memo[n]


print(fib(25), "en", appels, "appels")
appels = 0
print(fib_memo(25), "en", appels, "appels")
appels = 0
print(fib_memo(100), "en", appels, "appels")
```

Dans le terminal

```
$ python3 fibonacci.py
75025 en 242785 appels
75025 en 49 appels
354224848179261915075 en 151 appels
```

La version naïve recalcule `fib(23)` deux fois, `fib(22)` trois fois, `fib(21)` cinq fois… La version **mémoïsée** range chaque résultat dans un dictionnaire et ne le calcule qu'une fois : 49 appels au lieu de 242 785. `fib(100)` serait impossible en version naïve (des milliards d'années), et prend ici une fraction de seconde.

Le mot-clé `global` permet à une fonction de **modifier** une variable créée en dehors d'elle. À utiliser avec modération : ici, c'est juste pour compter.

4. Ranger son code en plusieurs fichiers

Quand un programme grandit, on range les fonctions dans un fichier à part, un **module**, qu'on importe comme `random` ou `math`. Chaque fichier `.py` est un module : son nom, sans le `.py`, est le nom à importer.

geometrie.py

```python
"""Quelques calculs de géométrie."""
import math


def aire_rectangle(longueur, largeur):
    return longueur * largeur


def aire_disque(rayon):
    return math.pi * rayon ** 2


if __name__ == "__main__":
    # Ces lignes ne s'exécutent que si on lance geometrie.py directement
    print("Tests de geometrie.py")
    print(aire_rectangle(3, 4) == 12)
    print(round(aire_disque(1), 2) == 3.14)
```

Dans le terminal

```
$ python3 geometrie.py
Tests de geometrie.py
True
True
```

jardin.py

```python
import geometrie

pelouse = geometrie.aire_rectangle(20, 8)
bassin = geometrie.aire_disque(2)
print(f"À tondre : {pelouse - bassin:.1f} m²")
```

Dans le terminal

```
$ python3 jardin.py
À tondre : 147.4 m²
```

Lancé directement, `geometrie.py` exécute ses tests. Importé par `jardin.py`, il ne fait que fournir ses fonctions : le bloc `if __name__ == "__main__":` est ignoré. Les deux fichiers doivent être dans le même dossier.

C'est une convention que vous verrez dans presque tous les programmes Python : les fonctions en haut, et le programme principal protégé par ce `if`.

5. Les compréhensions de liste

Une écriture courte, très utilisée en Python, pour fabriquer une liste à partir d'une autre :

Dans l'interpréteur Python

```
>>> [x * x for x in range(1, 6)]
[1, 4, 9, 16, 25]
>>> [x for x in range(20) if x % 3 == 0]
[0, 3, 6, 9, 12, 15, 18]
>>> [mot.upper() for mot in ["chat", "chien", "lapin"]]
['CHAT', 'CHIEN', 'LAPIN']
>>> [len(mot) for mot in "le chat dort".split()]
[2, 4, 4]
```

`[x * x for x in range(1, 6)]` se lit : « la liste des `x * x`, pour chaque `x` de 1 à 5 ». C'est exactement la boucle avec `append` de mardi, en une ligne. Pratique quand c'est simple. Dès que ça devient compliqué, revenez à la boucle : un code lisible vaut mieux qu'un code court.

6. Le débogueur de VS Code

Un bug, c'est un programme qui fait autre chose que ce que vous croyez. Le débogueur permet de le **regarder faire**, ligne par ligne, en voyant la valeur de chaque variable.

Prenons ce programme : la moyenne de 12, 8, 15 et 10 devrait être 11,25.

moyenne_bug.py

```
def moyenne(notes):
    total = 0
    for i in range(1, len(notes)):
        total += notes[i]
    return total / len(notes)


resultat = moyenne([12, 8, 15, 10])
print(resultat)
```

Dans le terminal

```
$ python3 moyenne_bug.py
8.25
```

8,25 : il manque 3 points, mais où ? Avec le débogueur :

1. **Poser un point d'arrêt** : cliquez dans la marge, à gauche du numéro de la ligne `total = 0`. Un rond rouge apparaît.
2. **Lancer en mode débogage** : `F5`, puis choisissez **Python Debugger: Debug Python File**. Le programme démarre et **s'arrête** sur la ligne du point d'arrêt.
3. **Avancer ligne par ligne** avec `F10` (*Step Over*). À gauche, le panneau **Variables** affiche `notes`, `total`, `i` et leur valeur, mise à jour à chaque pas.
4. Regardez `i` au premier passage dans la boucle : il vaut **1**, pas 0. La note 12, à l'indice 0, n'est jamais ajoutée. Le bug est trouvé : `range(1, …)` au lieu de `range(len(notes))`.

Touche	Action
`F5`	lancer le débogage, ou continuer jusqu'au prochain point d'arrêt
`F10`	exécuter la ligne et passer à la suivante
`F11`	entrer **dans** la fonction appelée sur cette ligne
`Shift+F5`	arrêter

Le débogueur ne trouve pas le bug à votre place, mais il remplace les suppositions (« `i` commence forcément à 0 ») par des faits.

Exercices du vendredi

V1. Le juste prix, côté ordinateur NIVEAU 1

Inversez le jeu d'hier : **vous** choisissez un nombre entre 1 et 100, et l'ordinateur le devine par dichotomie. À chaque proposition, vous répondez `+`, `-` ou `=`. L'ordinateur doit trouver en 7 coups au maximum. Que se passe-t-il si vous trichez ?

V2. Le tri à bulles NIVEAU 2

Vous l'avez écrit en bash : on parcourt la liste en échangeant deux voisins mal rangés, et on recommence tant qu'il y a eu au moins un échange. Écrivez `tri_bulles(liste)` en Python, et comptez le nombre d'échanges. Combien pour `[5, 4, 3, 2, 1]` ? Et pour une liste déjà triée ?

V3. La puissance NIVEAU 2

Écrivez une fonction **récursive** `puissance(x, n)` qui calcule `x` à la puissance `n` (entier positif), sans utiliser `**`. Indice : $x^0 = 1$, et $x^n = x \times x^{n-1}$.

V4. La somme des chiffres NIVEAU 2

Écrivez une fonction récursive `somme_chiffres(n)` : `somme_chiffres(1234)` vaut 1 + 2 + 3 + 4 = 10. Indice : `n % 10` donne le dernier chiffre, `n // 10` le nombre sans son dernier chiffre.

V5. Le palindrome récursif NIVEAU 2

Un mot est un palindrome si sa première et sa dernière lettre sont égales, **et** si ce qu'il y a entre les deux est un palindrome. Écrivez `est_palindrome(mot)` en version récursive. Quel est le cas de base ?

V6. Ma boîte à outils NIVEAU 2

Créez un module `outils.py` qui regroupe les meilleures fonctions de la semaine : `est_pair`, `est_premier`, `est_palindrome`, `demander_entier`, `moyenne`. Ajoutez un bloc `if __name__ == "__main__":` qui teste chacune d'elles en affichant `True` quand le résultat est bon. Puis écrivez un petit programme `menu.py` qui importe `outils` et propose à l'utilisateur de tester un nombre (pair ? premier ?).

V7. La chasse au bug NIVEAU 2

Ce programme doit afficher le plus long mot d'une phrase. Il répond `chat`. Trouvez le bug **avec le débogueur** (point d'arrêt, `F10`, panneau Variables), sans ajouter de `print`. Puis corrigez-le.

plus_long_bug.py

```python
def plus_long_mot(phrase):
    mots = phrase.split()
    meilleur = mots[0]
    for mot in mots:
        if len(mot) > len(meilleur):
            meilleur = mot
            return meilleur
    return meilleur


print(plus_long_mot("le chat regarde attentivement la souris"))
```

Dans le terminal

```
$ python3 plus_long_bug.py
chat
```

V8. Les tours de Hanoï NIVEAU 3

Trois piquets, des disques de tailles différentes empilés sur le premier, du plus grand en bas au plus petit en haut. Il faut tout déplacer sur le troisième, un disque à la fois, sans jamais poser un disque sur un plus petit. Écrivez `hanoi(n, depart, arrivee, intermediaire)` qui affiche les déplacements :

```
Déplacer le disque 1 de A vers C
Déplacer le disque 2 de A vers B
Déplacer le disque 1 de C vers B
...
```

L'idée récursive : pour déplacer `n` disques de A vers C, on déplace les `n - 1` du dessus de A vers B, puis le grand de A vers C, puis les `n - 1` de B vers C. Combien de déplacements pour 3 disques ? Pour 10 ? Trouvez la formule.

V9. La fusion de deux listes triées NIVEAU 3

Écrivez `fusion(a, b)` qui reçoit deux listes **déjà triées** et renvoie une seule liste triée, **sans** utiliser `sort` ni `sorted` : on avance dans les deux listes en même temps, en prenant à chaque fois le plus petit des deux éléments en tête. `fusion([1, 4, 9], [2, 3, 10, 12])` donne `[1, 2, 3, 4, 9, 10, 12]`. C'est le cœur du tri fusion, l'un des tris les plus rapides.

Mémo du vendredi

Je veux…	En Python
échanger deux valeurs	`a, b = b, a`
une liste triée (nouvelle)	`sorted(liste)`, `sorted(liste, reverse=True)`, `sorted(liste, key=len)`
trier sur place	`liste.sort()`
copier une liste	`copie = liste[:]`
fonction récursive	un cas de base, puis un appel sur un problème plus petit
mémoriser des résultats	un dictionnaire : `if n not in memo: memo[n] = …`
importer mon propre fichier	`import geometrie` (le fichier `geometrie.py`, dans le même dossier)
code exécuté seulement si on lance le fichier	`if __name__ == "__main__":`
liste en une ligne	`[x * x for x in range(10) if x % 2 == 0]`
mesurer un temps	`debut = time.perf_counter()` … `time.perf_counter() - debut`
déboguer	point d'arrêt dans la marge, `F5`, puis `F10` ligne par ligne